

PKCERT AI & Software Development Internship

Task 20: Feedforward Neural Network on Fashion-MNIST

Abdullah Amir

AI and Software Development
30 July 2026

This report covers the theory and the results. The full code and outputs live in the notebook `mnist_feedforward.ipynb`. **PyTorch 2.13.0 (CPU build)**, **torchvision 0.28.0** are used throughout, random seed **42** fixed for the train/validation split, model initialisation, and DataLoader shuffling. Tasks 18 and 19 built the network API fluency and training-loop mechanics used here; this task assembles both into one complete, rigorously evaluated pipeline on **Fashion-MNIST** (Xiao, Rasul & Vollgraf, 2017), the stated MNIST alternative, chosen because its confusion matrix is far more informative than plain MNIST's – clothing categories with genuine visual overlap, rather than MNIST's mostly-solved digit pairs.

Part A: Data Pipeline & Preprocessing

Three-way split. The official test set (10,000 images) is held out completely until final evaluation. The official 60,000-image training set is further split **50,000 (71.4%) train / 10,000 (14.3%) validation**, seed 42, so no hyperparameter or epoch-count decision ever touches the test set.

Normalization statistics, and the leakage they avoid – demonstrated, not just described. Computed from the 50,000-image training split only: mean = 0.2857, std = 0.3529. For comparison, computed from all 60,000 images (train+val together, the leaky way): mean = 0.2860, std = 0.3530 – a small but real difference (0.00029 in mean). Using the leaked statistics would mean every training-time normalization already encodes information about the exact 10,000 images later used for model selection; the gap would only grow on a smaller dataset or a less i.i.d. split, so the principle matters even when today's numeric impact is small.

Pipeline shapes, printed at every stage: raw PIL image 28×28 (mode L) $\rightarrow$ after `ToTensor()`: $(1, 28, 28) \rightarrow$ after flatten: $(784,)$ $\rightarrow$ batched tensor: $(128, 784)$, labels $(128,)$.

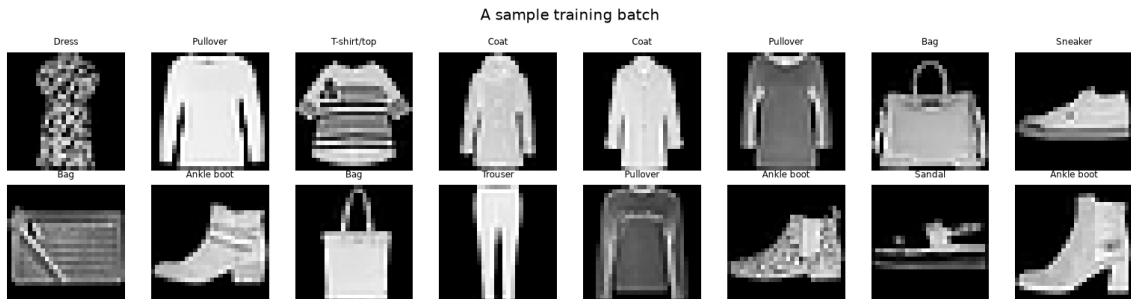


Figure 1: A sample training batch with true labels.

Class distribution. Reasonably but not perfectly balanced, as expected from a non-stratified random split: the training split ranges from 4,926 to 5,055 images per class (of 5,000 expected if perfectly even); validation and test are similarly close.

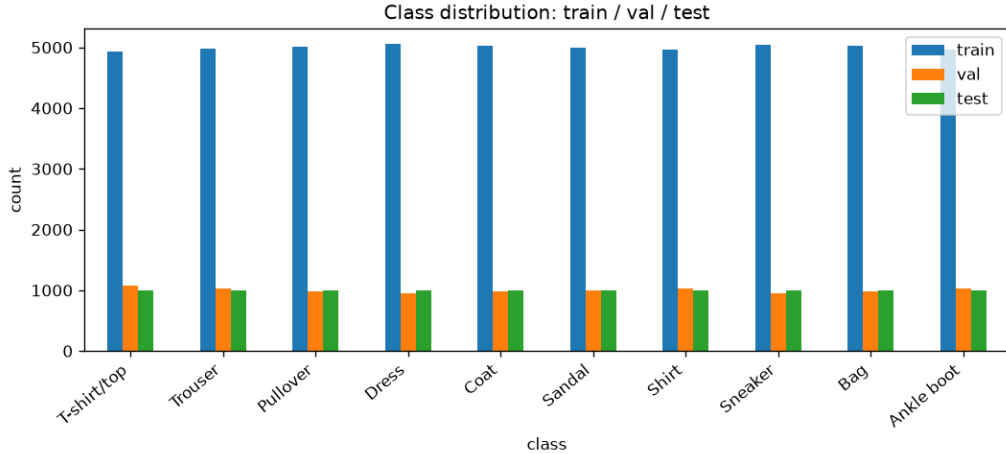


Figure 2: Class counts across the train / validation / test splits.

Part B: Model Architecture & Design Justification

Architecture: $784 \rightarrow 256 \rightarrow 128 \rightarrow 10$ (3 fully-connected layers, 2 hidden). 256 (roughly a third of the 784-dimensional input) gives the first layer enough width to form a broad bank of pixel-pattern detectors without the parameter count exploding ($784 \times 256 = 200,704$ weights already dominates the model); 128 narrows that by half, compressing toward the 10-class output through an intermediate representation. Two hidden layers, not more, was deliberate for a purely fully-connected network (no weight sharing, unlike a CNN) on a 50,000-image training set: enough depth to compose non-linear features, not so much that it overfits before the data can constrain it.

Activation: ReLU. Sigmoid and tanh both saturate: their derivatives approach 0 as $|x|$ grows (sigmoid'(x) peaks at only 0.25; tanh'(x) peaks at 1.0 but still decays fast). Backpropagation multiplies the upstream gradient by each layer's local derivative, so stacking two such layers multiplies together two numbers usually well below 1, shrinking the gradient reaching the first layer before training has even moved it away from initialisation. ReLU's derivative is exactly 1 for any positive input, so it passes gradient through unattenuated on its active side – the deeper the stack, the more this matters.

Logits+CrossEntropyLoss vs softmax+NLLoss, confirmed numerically equal.

Formulation	Loss on a sample batch
<code>CrossEntropyLoss(logits, y)</code>	2.304028
<code>NLLoss(log_softmax(logits), y)</code>	2.304028

Table 1: Identical to 6 decimal places – CrossEntropyLoss literally computes `log_softmax` then NLLoss internally.

Both formulations compute the mathematically identical quantity. The logits+CrossEntropyLoss form is numerically preferred because it computes $\log(\text{softmax}(z))$ directly via the log-sum-exp identity ($\log \text{softmax}(z)_i = z_i - \max(z) - \log \sum_j \exp(z_j - \max(z))$) in one fused, stable operation, rather than computing softmax as an intermediate tensor (risking underflow to exactly 0.0 for a confidently wrong logit) and taking its log separately.

Parameter count, by hand and verified.

Layer	Weights	Biases	Total
784 $\rightarrow$ 256	200,704	256	200,960
256 $\rightarrow$ 128	32,768	128	32,896
128 $\rightarrow$ 10	1,280	10	1,290
Hand-calculated total			235,146
<code>sum(p.numel() for p in model.parameters())</code>			235,146

Table 2: Exact match.

Part C: Training, Evaluation & Experiments

Optimizer: Adam(lr=0.001), batch size 128, 20 epochs. Task 19’s own controlled comparison found tuned SGD+momentum beating Adam on the small, 4-feature Palmer Penguins network – attributed there to Adam’s adaptive scaling having no uneven-gradient pathology to correct on such a small, well-conditioned problem. This network is a different regime: 784 input dimensions of very unevenly informative pixels (edge pixels are almost always 0, centre pixels carry most of the signal), closer to the sparse/uneven-gradient case Task 19’s worked example showed Adam handling better than plain SGD. Batch size and epoch count were chosen as a standard, computationally light starting point; the loss curves below confirm this was enough epochs to see validation loss plateau.

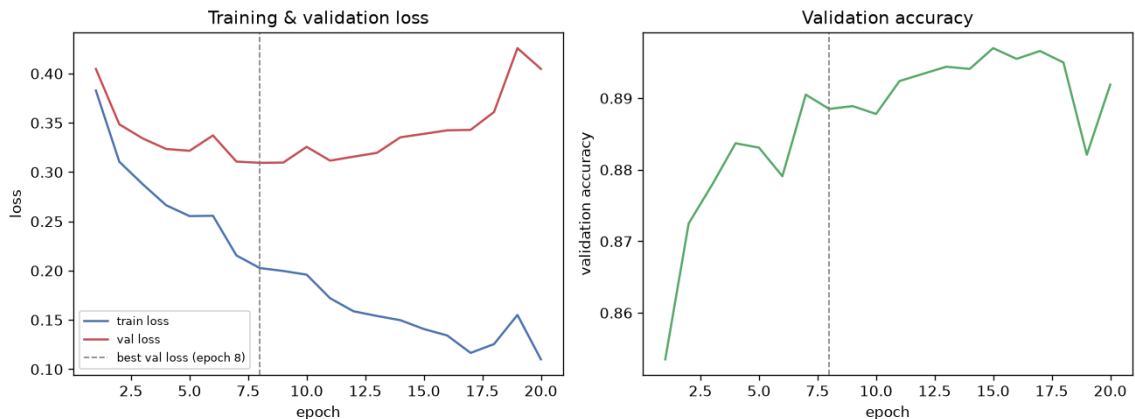


Figure 3: Training/validation loss and validation accuracy across 20 epochs.

Validation loss is lowest at epoch 8 (0.3094) and rises steadily afterward (0.4048 at epoch 20) – mild overfitting begins around epoch 8. Validation *accuracy*, however, keeps drifting upward with noise until around epoch 15, since accuracy is a coarser, thresholded measurement than the continuous loss and can keep improving slightly even as the model’s confidence calibration (which loss penalises directly) starts to degrade – the two curves need not peak together.

Metric (test set, 10,000 images)	Value
Accuracy	0.8870
Macro-Precision	0.8880
Macro-Recall	0.8870
Macro-F1	0.8873

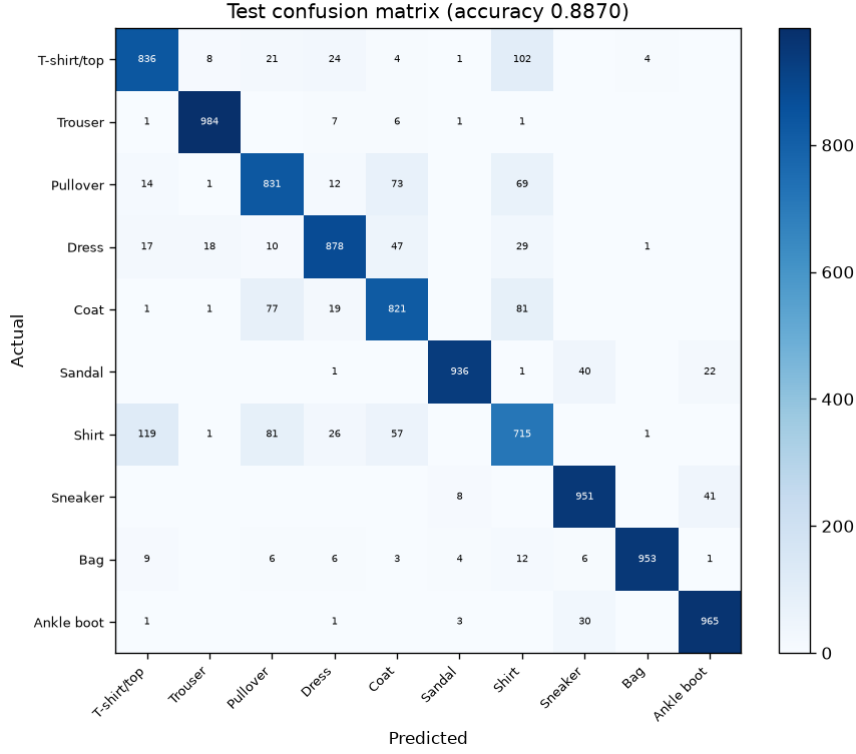


Figure 4: Test confusion matrix, all 10 classes.

Shirt and T-shirt/top are the most confused pair: 119 true-Shirt images predicted T-shirt/top, 102 the reverse. Pullover/Coat/Shirt also cross-confuse substantially (73, 81, 77, 81 off-diagonal counts among the three). All are upper-body garments with overlapping silhouettes at 28×28 greyscale resolution, with no colour or texture cue available to separate them – exactly the kind of confusion a purely fully-connected, spatially-blind network is expected to make.

Controlled ablation: network depth

Configuration	Parameters	Test accuracy
0 hidden layers (linear)	7,850	0.8417
1 hidden layer (256)	203,530	0.8797
2 hidden layers (256, 128)	235,146	0.8833

Table 3: 12 epochs each, identical seed, optimizer and batch size.

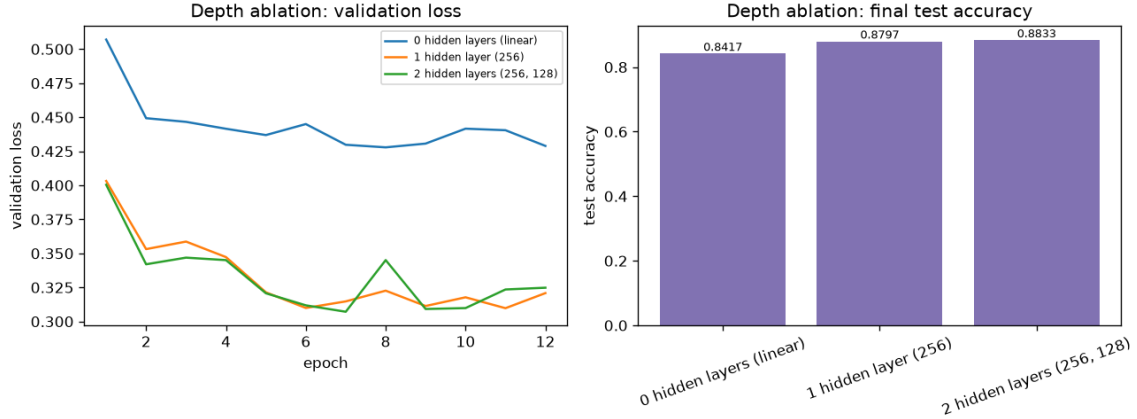


Figure 5: Validation loss and final test accuracy across the depth ablation.

Adding the first hidden layer (0→1) is worth **+3.80** points of test accuracy for 195,680 more parameters; adding the second (1→2) is worth only **+0.36** points for 31,616 more parameters – returned to in Part D.

Model persistence, verified. `torch.save(model.state_dict(), ...)`, reloaded into a freshly-initialised model instance with `load_state_dict`: predictions on a held-out batch were confirmed **exactly identical** to the original model’s, element for element.

Part D: Analysis & Documentation

Architecture	Params	Test Acc	Macro-P	Macro-R	Macro-F1
Linear (0 hidden)	7,850	0.8417	–	–	–
1 hidden layer (256)	203,530	0.8797	–	–	–
2 hidden layers (256,128), main model	235,146	0.8870	0.8880	0.8870	0.8873

Table 4: Precision/Recall/F1 computed for the main (20-epoch) model; the ablation configurations (12 epochs) are scored on accuracy only, sufficient to isolate the effect of depth. The main model’s 2-hidden-layer test accuracy (0.8870) differs slightly from the ablation’s identical architecture (0.8833) purely because of the different epoch budget (20 vs 12).

Counter-intuitive finding. Going from one hidden layer to two barely moved test accuracy (+0.36 points) **despite adding nearly as many parameters as the entire first hidden layer contains** (31,616 new parameters vs 200,960 in the whole first layer). The likely mechanism: a single 256-unit hidden layer already has more than enough capacity to separate Fashion-MNIST’s 10 classes given only 784 raw pixel inputs – the representational bottleneck was never depth, it was always the fact that raw pixel intensities, even linearly recombined by one hidden layer’s worth of ReLU units, already capture most of the separable signal a fully-connected (no weight-sharing, no spatial structure) network can extract. A second hidden layer adds parameters faster than it adds separable signal once the first layer has already captured the easy wins – the general shape of diminishing returns to depth in a plain MLP, and part of why convolutional architectures, which use depth to build spatial hierarchies rather than just stack more dense capacity, are the actual next step past this kind of network for image data.

A concrete implementation issue. The normalization-leakage check in Part A was not originally part of the plan – it was added after the numbers needed independent verification. An

early version of this pipeline computed the normalization mean/std from `raw_train_full.data` directly (all 60,000 images) before the train/val split existed in the code, which is exactly the leakage Part A warns against. It was caught by writing the leakage comparison *deliberately*: computing the statistic both ways (train-only vs full-60k) and checking that they should differ once the split existed – had a bug left the "train-only" computation silently identical to the full-set one, that agreement itself would have been the signal that it was still touching data outside the training split. The fix was a one-line reordering: compute `train_images` from `raw_train_full.data[train_subset.indices]` specifically, after the split is defined, not from the full tensor before it – easy to get backwards if the split and the normalization code are not read together as one dependent unit.

Reproducibility. PyTorch 2.13.0 (CPU build), torchvision 0.28.0. Random seed 42 fixed for the train/val split, model initialisation, and DataLoader shuffling. Optimizer: Adam, $\text{lr}=10^{-3}$. Batch size: 128. Epochs: 20 (main model), 12 (each ablation configuration – a shorter, deliberately lighter budget, past the point where the depth ordering had stabilised).

Conclusion. Every claim in this report was checked against the pipeline’s own output rather than assumed: the leakage a full-dataset normalization would cause was measured, not just described; both loss formulations were confirmed numerically identical rather than asserted equivalent; the hand-calculated parameter count was verified against PyTorch’s own count; and the value of network depth was measured directly rather than assumed monotonic – which is exactly what surfaced the diminishing-returns finding above. The result is an 88.70% test-accuracy fully-connected (no convolutions) classifier on Fashion-MNIST, with every design choice traceable to a number from this pipeline’s own run.

The notebook and README are on GitHub: <https://github.com/AbdullahAmir06/ai-internship-abdullahamir>